

SONOPANT DANDEKAR COLLEGE, PALGHAR

(Affiliated to University of Mumbai)

Department of Computer Science

MINI PROJECT DOCUMENTATION

CodeDropz

Paste. Generate. Share. -- A Cross-Device Code, Text & File Sharing Platform

Submitted in partial fulfilment of the requirements for the
Degree of Bachelor of Science in Computer Science (B.Sc. CS)
Third Year -- Semester V, Academic Year 2026-27

Submitted By

Adarsh Jayprakash Mishra

Under the Guidance of

Prof. _____

Table of Contents

2. Problem Statement	3
3. Feasibility Study	5
4. Requirement Engineering	7
5. Software Development Life Cycle (SDLC) Planning	9
6. System Model using UML (9 Diagrams)	12
7. System Architecture Design	21
8. Implementation, Testing, Deployment & Evaluation.....	23

2. Problem Statement

Every developer, student and everyday computer user eventually runs into the same small but recurring annoyance: a piece of code, a paragraph of notes, or a handful of files needs to move from one device to another -- a laptop to a phone, a lab computer to a personal machine, or one browser tab to a friend's screen -- and there is no fast, clean, account-free way to do it. CodeDropz was conceived to close exactly this gap. This section frames the problem from three complementary angles: the professional/industry angle, the everyday social angle, and the institutional angle relevant to a college or shared-computing environment.

2.1 Industry Perspective

In a professional software development setting, engineers frequently need to move a short log excerpt, a configuration snippet, a stack trace, or a small set of build artifacts between machines -- for example from a headless CI runner to a personal laptop, or from a client's remote desktop to a developer's own system. The common workarounds (emailing oneself, pasting into a chat app, or creating a public Gist) either introduce delay, break code formatting, or leave the content sitting permanently on a third-party server. For a QA or automation engineer in particular, sharing a failing test's console output or a small screenshot bundle with a teammate on another device is a task that happens many times a day and deserves a near-instant, disposable solution.

2.2 Social Perspective

Outside of work, ordinary users face the same friction at a smaller scale: a student copying an assignment snippet from a college desktop to their own phone, a person moving a saved Wi-Fi password or a set of holiday photos between two personal devices, or a group of friends wanting to pass a small file around without installing a platform-specific tool such as AirDrop (Apple-only) or Nearby Share (Android-only). Chat applications used as a personal notepad-to-self are a very common but poor substitute -- they compress images, wrap long lines of code awkwardly, and retain the content indefinitely inside a chat history that was never meant to be a file store.

2.3 Institutional Perspective

In a college computer lab, cyber cafe, or any shared/public computer, creating yet another account just to move a file is both inconvenient and a privacy concern -- the account and its recovery email persist on a machine the user does not own. Institutions also value tools whose data does not linger indefinitely on shared hardware. An anonymous, expiring, login-free transfer mechanism is therefore particularly well suited to lab environments, examination centres, and any setting where a person is only a temporary occupant of the device they are using.

2.4 Limitations of Existing Approaches

The table below summarises the tools people currently reach for and the specific limitation each one carries with respect to a quick, cross-device, self-cleaning transfer.

Common Approach	Limitation
WhatsApp / Telegram (message to self)	Breaks code formatting and indentation, adds unwanted line wraps, and stores the content forever in chat history.
Email to self	Slow round-trip, cluttered inbox, and requires signing in on both the sending and receiving device.
GitHub Gist / Pastebin	Requires an account; content remains public or permanent unless manually deleted.
USB drive / cable	Physical dependency -- not always on hand, and offers no support for mobile phones.
Cloud drives (Drive, Dropbox)	Heavy login flow and sync overhead for what is often just a 20-line snippet.
AirDrop / Nearby Share	Platform-locked -- Apple-only or Android-only, no cross-platform / browser option.

2.5 Problem Definition

There is currently no fast, login-free, self-destructing, and truly cross-platform way to move a code snippet, a block of text, or a small set of files between two devices. CodeDropz is proposed as a web-based solution that lets a sender paste content or attach files, choose an expiry and an access-limit, and receive a short share code; a receiver on any other device enters that code and instantly retrieves the content, after which it is automatically and permanently deleted from the server.

3. Feasibility Study

Before committing development effort, the proposed system was evaluated along the three standard feasibility dimensions -- technical, economic, and operational -- to confirm that CodeDropz could realistically be built, afforded, and used within the scope of a college mini-project.

3.1 Technical Feasibility

The technology choices behind CodeDropz are all mature, well-documented, and freely available, which keeps technical risk low:

- React 19 with TypeScript gives a type-safe, component-based frontend with a very large ecosystem and community support.
- TanStack Start / TanStack Router provides file-based routing and server-side rendering without needing a hand-rolled Node/Express backend.
- A managed PostgreSQL platform (with Row Level Security and PL/pgSQL functions) removes the need to provision or patch a database server manually.
- Object storage, hourly scheduled cleanup jobs, Vercel hosting, and GitHub-based deployment are all offered on generous free tiers suited to a student project.

Given that the required skills (React, TypeScript, SQL, basic cloud deployment) were already within reach and all supporting platforms are documented and widely used, the project was judged technically feasible.

3.2 Economic Feasibility

CodeDropz was built and hosted entirely on free-tier services during development and demonstration, which keeps the direct monetary cost close to zero:

Cost Head	Estimated Cost	Notes
Frontend hosting (Vercel)	Rs. 0	Free tier is sufficient for a low-traffic student project.
Managed PostgreSQL + storage	Rs. 0	Free tier covers table storage, RLS and function execution.
Domain / SSL	Rs. 0	Default *.vercel.app subdomain with automatic HTTPS.
Development effort	Developer time only	No paid tooling or licences were required.

Because there is no recurring licence fee and no dedicated server to maintain, the ongoing cost of running CodeDropz at a small scale remains negligible, and the project comfortably fits within a zero-budget academic setting.

3.3 Operational Feasibility

Operational feasibility considers how easily the system will be adopted and kept running day to day:

- No account creation is required on either the sending or the receiving side, which removes the single biggest barrier to casual adoption.
- The interface works in any modern browser on any operating system, so no installation step is needed.
- The hourly cleanup job automatically removes expired records without any manual administration.
- Row Level Security and the atomic `retrieve_share()` function keep the data model self-enforcing, so there is no separate admin panel to operate or secure.

Because the system requires no ongoing manual intervention and its usage pattern (paste, generate a code, retrieve, auto-delete) mirrors habits users already have from tools like Pastebin, it was judged operationally feasible for both individual and small-group use.

3.4 Feasibility Summary

Feasibility Type	Verdict	Key Reason
Technical	Feasible	Mature, well-supported, free-tier-friendly stack.
Economic	Highly Feasible	Zero direct cost using free hosting and managed backend tiers.
Operational	Feasible	Login-free, browser-based, self-maintaining via automated cleanup.

4. Requirement Engineering

Requirement engineering for CodeDropz combined informal observation of real transfer pain-points (described in the Problem Statement) with a review of comparable tools such as Pastebin, Gist, and file-drop services, to arrive at a focused set of functional and non-functional requirements.

4.1 Requirement Gathering Approach

Because CodeDropz is a self-directed mini-project rather than a client-commissioned system, requirements were gathered through: (a) first-hand observation of everyday cross-device transfer friction, (b) a comparative study of the strengths and weaknesses of existing tools (summarised in Section 2.4), and (c) iterative refinement during development, where early feedback from test usage on multiple devices shaped later features such as custom codes and folder uploads.

4.2 Functional Requirements

1. FR-1: The system shall allow a user to paste text or code of any language into an editor without account creation.
2. FR-2: The system shall allow a user to upload one or more files, or an entire folder with its structure preserved.
3. FR-3: The system shall let the sender choose an expiry duration from preset options (10 minutes, 30 minutes, 1 hour, 1 day, 7 days, 1 month).
4. FR-4: The system shall let the sender set a maximum access (retrieval) limit -- 1, 5, 10, or a custom number.
5. FR-5: The system shall generate a random, unambiguous 6-character share code, or accept a custom 4-12 character code chosen by the sender.
6. FR-6: The system shall produce a shareable direct link (/r/<CODE>) and a scannable QR code for the generated share.
7. FR-7: The system shall allow a receiver to enter the share code (or open the link / scan the QR) on any device and view the shared content.
8. FR-8: The system shall let the receiver copy the text, download it as a .txt file, or download all files as a folder-preserving .zip archive.
9. FR-9: The system shall automatically and permanently delete a share's data once its access limit is reached or its expiry time has passed.
10. FR-10: The system shall display live indicators of remaining views and time-to-expiry on the receive page.
11. FR-11: The system shall reject a custom code that is already in use and prompt the sender to choose another.

4.3 Non-Functional Requirements

Category	Requirement
Usability	Zero-friction UI -- no sign-up, no email verification, minimal steps from paste to shareable code.
Security	Reads are only possible through a single access-controlled function; Row Level Security prevents direct table reads; no secret key ships to the browser.
Performance	Share retrieval should complete within a couple of seconds on a typical broadband or mobile connection.
Reliability	Concurrent retrieval of the same code must be handled safely so the access limit cannot be over-consumed (row locking).
Portability	The application must work identically on desktop and mobile browsers without a native app.
Maintainability	Expired data should be purged automatically (hourly job) without manual administrator effort.
Scalability	The stateless frontend and managed backend should scale horizontally with user load without redesign.

4.4 Hardware Requirements (Development Environment)

Component	Minimum Specification
Processor	Dual-core 2.0 GHz or higher
RAM	8 GB or higher (for a smooth Vite dev-server experience)
Storage	5 GB free disk space for project, dependencies and build cache
Internet Connection	Required for package installation, backend calls and deployment

4.5 Software Requirements

Component	Technology / Version
Frontend Framework	React 19, TypeScript 5.8
Build Tool	Vite (with Hot Module Replacement)
Full-stack Framework	TanStack Start / TanStack Router v1
Styling	Tailwind CSS v4, shadcn/ui, Radix UI primitives
Backend / Database	Managed PostgreSQL with Row Level Security
Object Storage	Cloud object storage bucket (share-files)
Hosting	Vercel
Version Control	Git and GitHub

5. Software Development Life Cycle (SDLC) Planning

5.1 SDLC Model Selected

CodeDropz was developed using the Incremental / Iterative model. The project began with a landing page and a mock MVP running purely on browser storage, and each subsequent increment added one coherent capability -- a real backend, file and folder uploads, expiry presets, custom codes, UI polish, and finally SEO and deployment -- with each increment tested before the next one began. This model was chosen over a strict Waterfall approach because the exact shape of the storage schema and the file-upload flow only became clear once the send/receive flow had been used first-hand, and an iterative approach allowed those lessons to feed directly into the next increment.

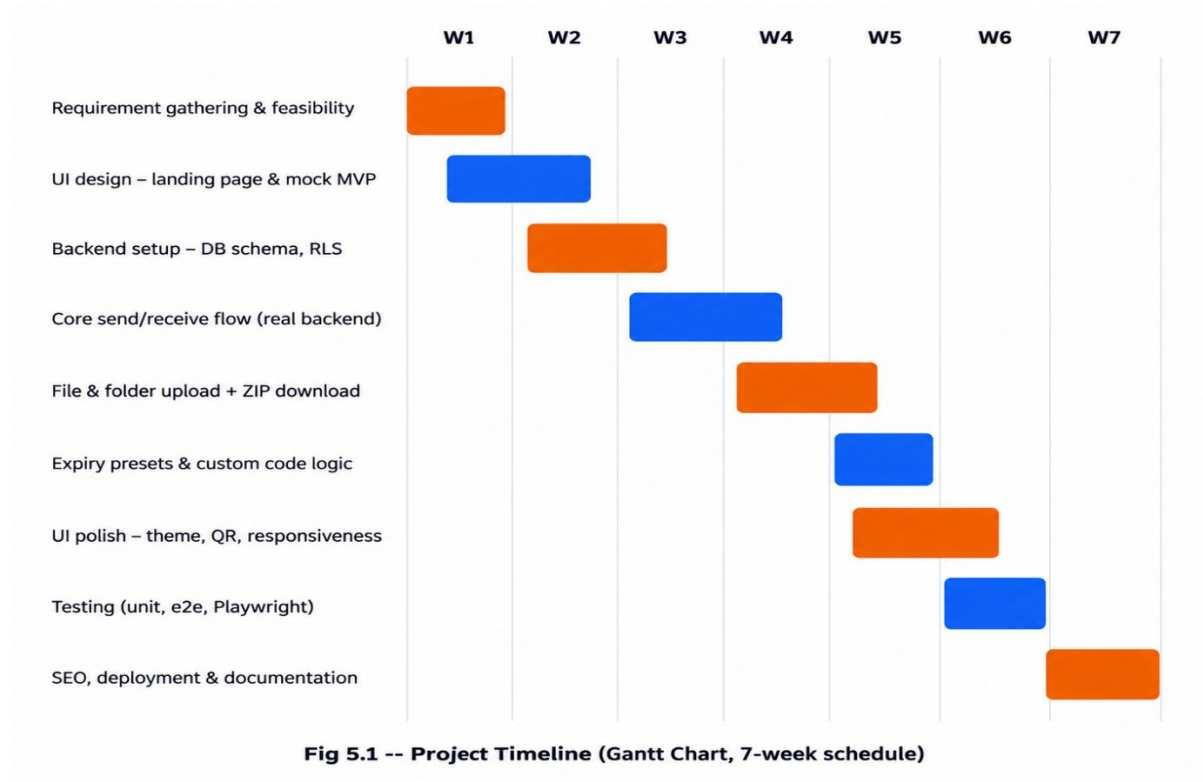
5.2 Work Breakdown Structure (WBS)

The project was broken down into the following major work packages, closely mirroring the actual development journey followed:

#	Work Package	Key Deliverable(s)
1	Requirement gathering & feasibility	Problem statement, feasibility study, finalised feature list
2	Landing page + mock MVP	Marketing site and a working send/receive flow backed by localStorage
3	Design refinements	Dark + orange default theme, blocking theme script, standardised 6-char codes
4	Real backend integration	shares table, RLS policies, atomic retrieve_share() function, hourly cleanup job
5	File & folder sharing	Object-storage uploads with metadata, per-file and folder-preserving ZIP downloads
6	Expiry expansion	Preset expiry options (10 min to 1 month) replacing free-form input
7	Custom share codes	Random/Custom toggle with validation and duplicate-code handling
8	Scope trimming	Removal of an admin panel and sender-side delete to keep the product login-free
9	UX fixes	Action buttons repositioned above the snippet; receive-page loading bug fixed
10	SEO, testing & launch	Metadata, JSON-LD, sitemap, Playwright checks, deployment to Vercel

5.3 Project Timeline

The work packages above were scheduled across a 7-week development cycle, with each task allotted realistic time for design, implementation and verification before the next began. The Gantt chart below visualises this schedule.



5.4 Milestones

Week	Milestone	Status
Week 2	Feasibility study & requirements signed off	Completed
Week 4	Working send/receive flow on a real backend	Completed
Week 5	File, folder and ZIP download support complete	Completed
Week 6	Custom codes and full UI polish complete	Completed
Week 7	Testing, SEO and production deployment complete	Completed

6. System Model using UML

This section models CodeDropz using the nine standard UML diagrams -- Use Case, Class, Object, State Chart, Activity, Sequence, Collaboration (Communication), Component, and Deployment. Together these diagrams describe the system's actors and behaviour, its static structure, a live instance of that structure, the lifecycle of its central data object, the flow of the two main use cases (create and retrieve), and the physical arrangement of its software components across client and server.

6.1 Use Case Diagram

The use case diagram identifies two human actors -- the Sender and the Receiver -- and one system actor, the Scheduler, which represents the hourly cleanup job. The Sender's use cases (paste/upload content, set expiry and access limit, choose a code, generate the share) sit alongside the Receiver's use cases (enter the code, view content, copy or download). The "Auto-Delete" use case includes "Validate Expiry & Access Limit", reflecting that every retrieval first checks whether the share is still valid before it can be deleted or served.

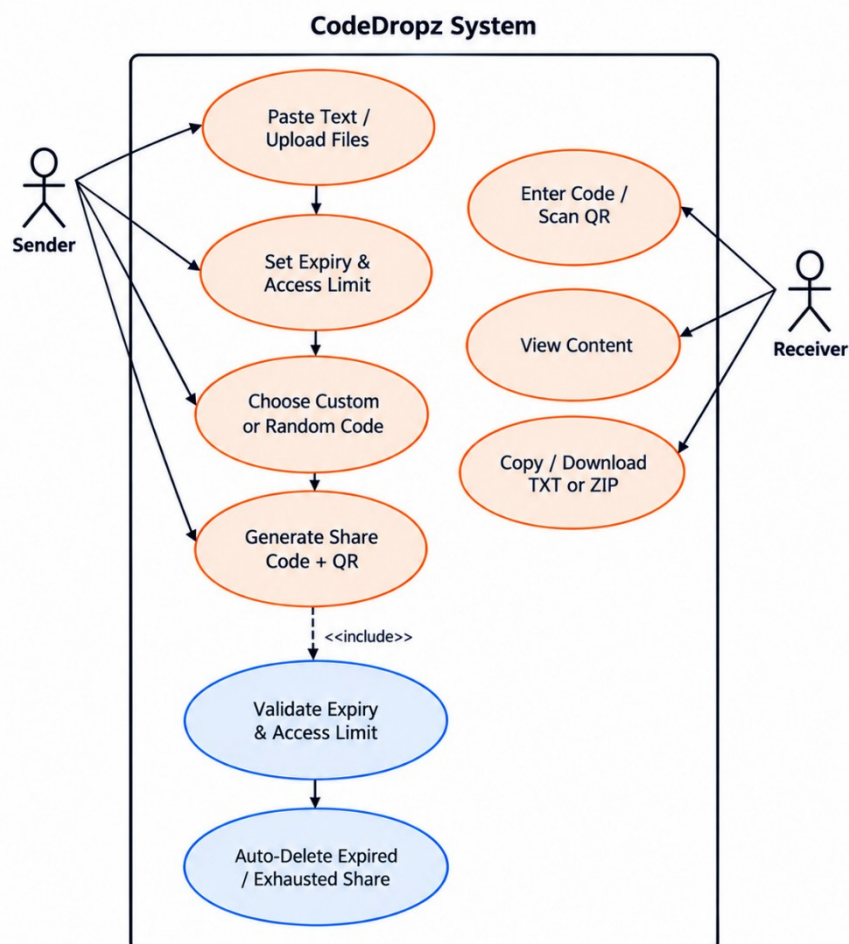


Fig 6.1 -- Use Case Diagram

6.2 Class Diagram

The class diagram captures the static structure of the system. The central Share class holds the code, content, expiry, and access-control fields, and is composed of one or more FileMeta objects when files are attached. The ShareStore class represents the application's data-access layer, used by both the ProductApp (send flow) and ReceivePage (retrieve flow) components on the frontend, which in turn rely on the Database class for the atomic retrieve_share() and cleanup_expired_shares() operations.

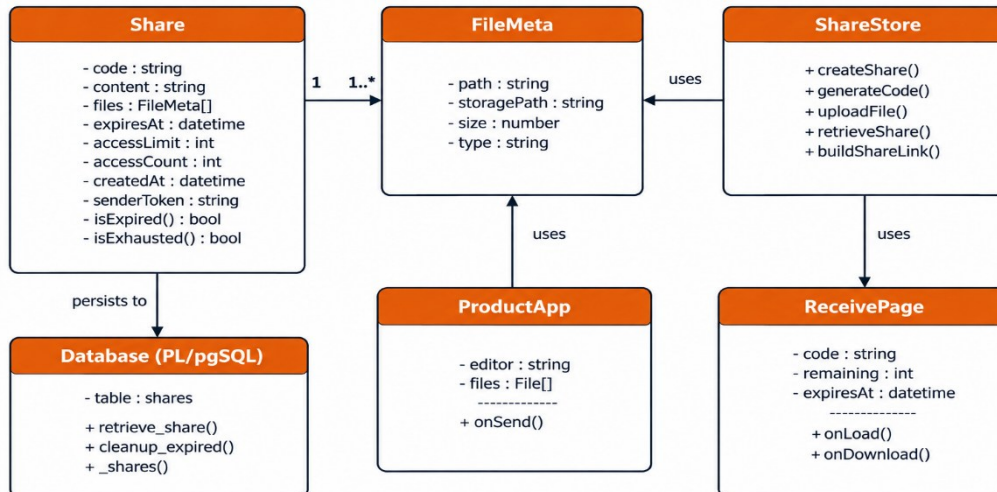


Fig 6.2 -- Class Diagram

6.3 Object Diagram

The object diagram shows a concrete snapshot of a Share instance at run time -- share1 -- together with two associated FileMeta objects, f1 and f2. It illustrates how the abstract Class Diagram is realised as real data during an actual transfer: a 6-character code, a short piece of pasted content, an expiry timestamp, and an access count that increases with every retrieval.

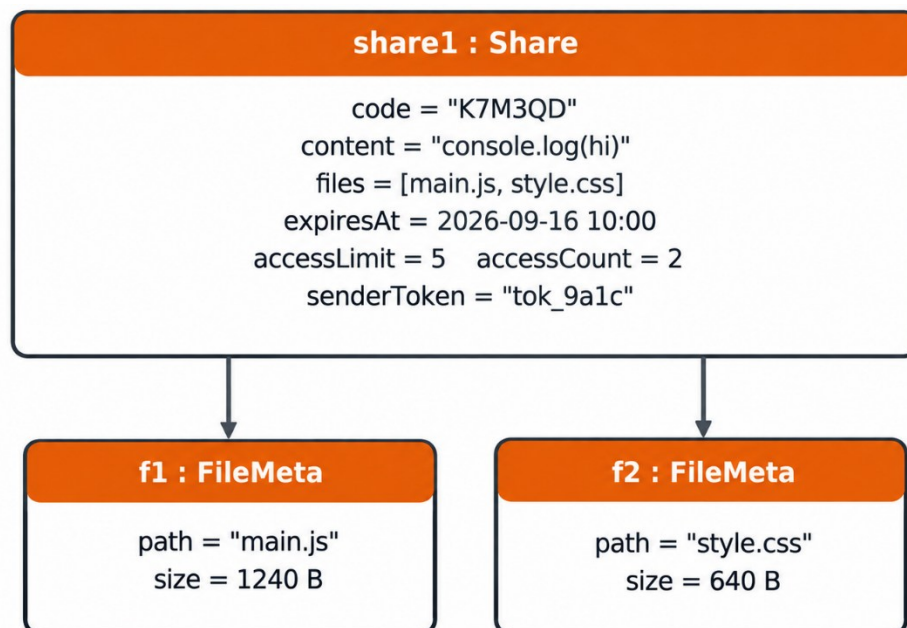


Fig 6.3 -- Object Diagram (a live share instance)

6.4 State Chart Diagram

The state chart diagram traces the lifecycle of a single Share record from creation to deletion. A share moves from Created to Active as soon as its row is inserted; from Active it can transition to Exhausted (its access_count has reached its access_limit) or, once its expiry timestamp passes, to Expired. Both of these terminal-ish states lead to Deleted, either immediately via the atomic retrieve_share() function on the next access attempt, or via the hourly cleanup_expired_shares() job.

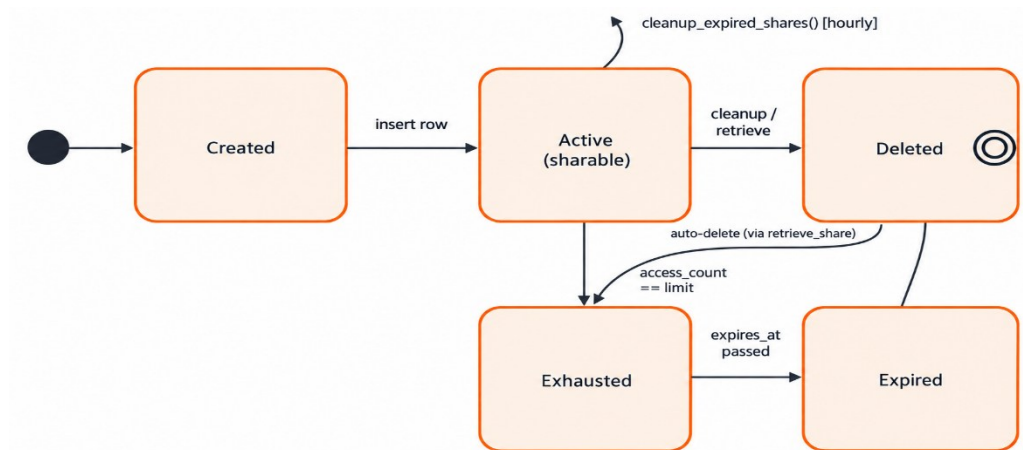


Fig 6.4 -- State Chart Diagram (lifecycle of a Share)

6.5 Activity Diagram

The activity diagram details the step-by-step flow followed when a sender creates a new share: pasting content or selecting files, client-side validation, uploading any files to object storage, inserting the database row, and then branching on whether that insert succeeded. A successful insert proceeds to generate the share code and display it with its QR code; a failed insert triggers a rollback of any files that were already uploaded, so no orphaned files remain in storage.

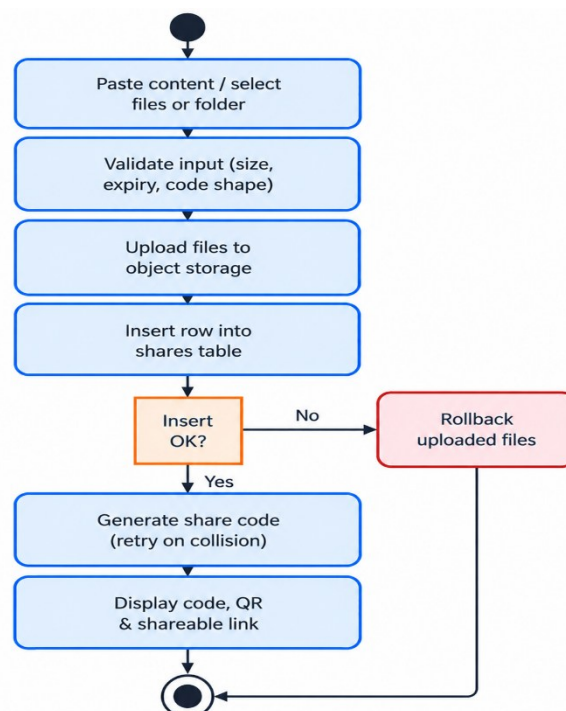


Fig 6.5 -- Activity Diagram (Create-Share flow)

6.6 Sequence Diagram

The sequence diagram illustrates the retrieve-share flow across time. The Receiver's action triggers the ReceivePage UI to call the ShareStore library, which issues a single RPC call to the retrieve_share() database function. That function locks the row, validates expiry and access count in one atomic step, and returns the content and file list, after which the UI fetches the actual file objects from storage and renders the result -- all before offering Copy and Download actions to the Receiver.

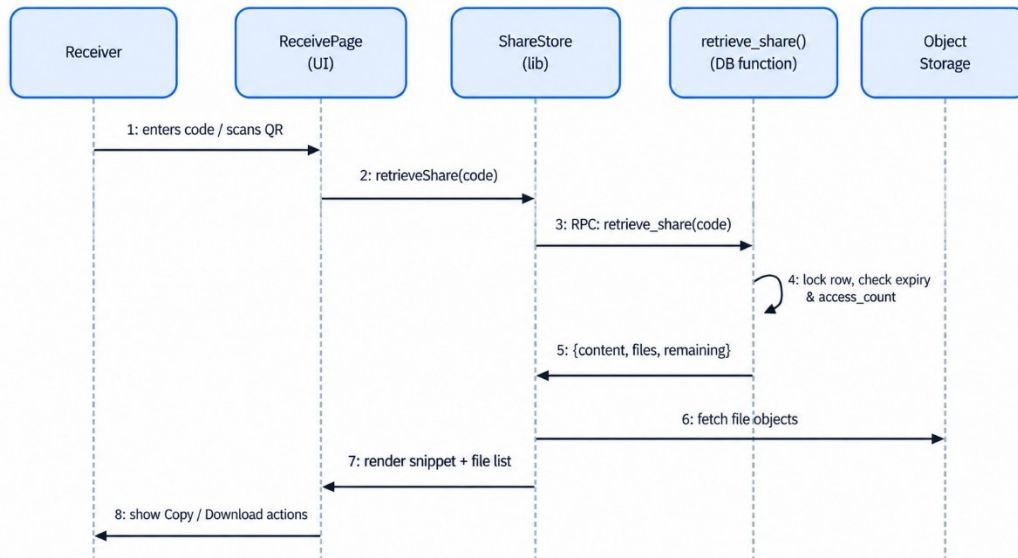


Fig 6.6 -- Sequence Diagram (Retrieve-Share flow)

6.7 Collaboration / Communication Diagram

This diagram presents the same retrieve-share interaction as Fig 6.6, but organised around the collaborating objects rather than a timeline, with each message numbered in the order it occurs. It emphasises which components talk to which -- for instance, that the ReceivePage never talks to the database or storage directly, but always goes through the ShareStore layer.

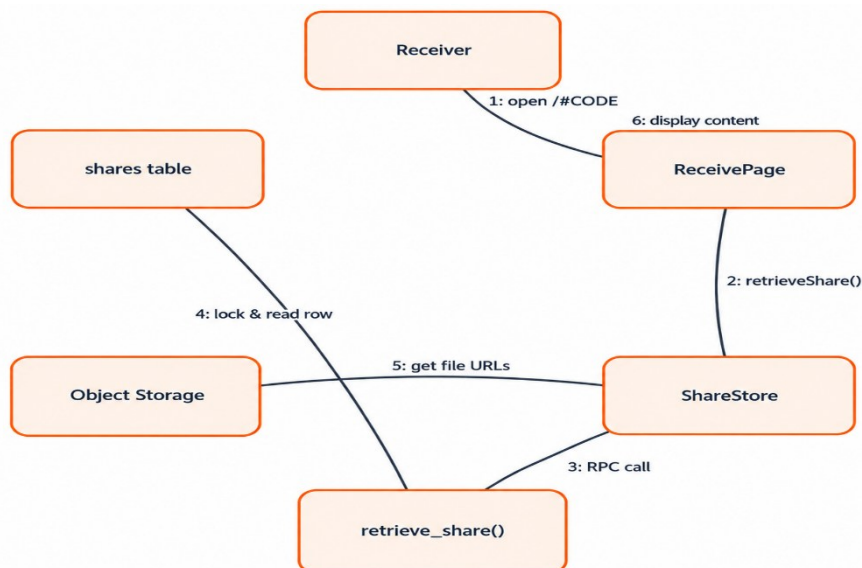


Fig 6.7 -- Collaboration / Communication Diagram

6.8 Component Diagram

The component diagram groups the system's building blocks into a Frontend package (Navbar/ThemeControls, ProductApp, ReceivePage, and the share-store.ts data layer) and a Backend package (the retrieve_share() and cleanup_expired_shares() functions, the shares table, and the share-files storage bucket). The two packages communicate exclusively over REST/RPC calls issued by the generated backend client, keeping the frontend free of any direct database credentials.

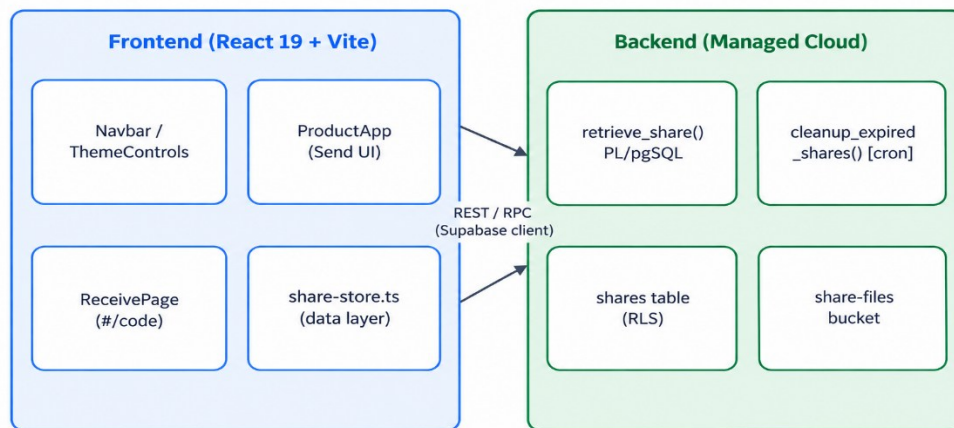


Fig 6.8 -- Component Diagram

6.9 Deployment Diagram

The deployment diagram shows the physical/runtime nodes involved: the Client Browser running the React single-page application, the Vercel Edge node serving static assets and SSR handlers over HTTPS, and the managed cloud backend split into a PostgreSQL node (table and functions) and an Object Storage node (the share-files bucket), reached over SQL/RPC and the Storage API respectively.

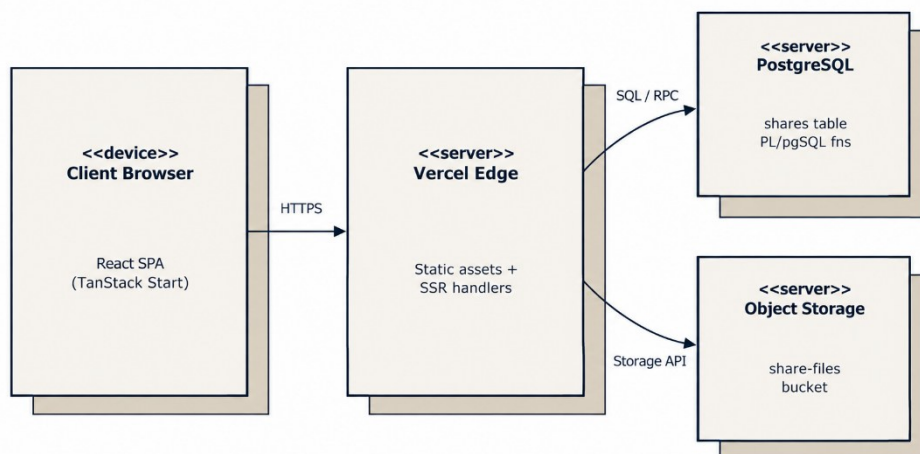


Fig 6.9 -- Deployment Diagram

7. System Architecture Design

CodeDropz follows a clean three-tier architecture that separates presentation, application logic, and data/storage concerns. This separation keeps the frontend free of any direct database access and allows the backend's security rules to be enforced in exactly one place -- the database layer itself.

7.1 Frontend Architecture

The frontend is a React 19 single-page application built with TypeScript and Vite, routed with TanStack Router on top of the TanStack Start full-stack framework, which also provides server-side rendering for the landing page. UI primitives come from shadcn/ui and Radix UI, styled with Tailwind CSS v4 theme tokens; sonner handles toast notifications, jszip assembles client-side ZIP downloads, and the qrcode package renders the share-link QR code. The three main routes are the landing page and ProductApp (index.tsx), the receive page (r.\$code.tsx), and the shared application shell (__root.tsx) that injects the blocking theme script, global metadata, and the toaster.

7.2 Backend Architecture

The backend is a managed PostgreSQL platform rather than a hand-built server. The shares table stores every drop's code, content, file metadata, expiry, and access counters, protected by Row Level Security so that anonymous clients may insert a new share but can never freely SELECT existing rows. All reads instead go through the retrieve_share(code) PL/pgSQL function, declared SECURITY DEFINER, which locks the target row, checks expiry and access count in a single transaction, increments the access count, and deletes the row automatically when the last permitted access has been used. A second function, cleanup_expired_shares(), runs on an hourly schedule to remove any shares that expired without ever being fully consumed.

7.3 API / Data-Flow Structure

Because CodeDropz has no custom REST server, its "API" is the set of RPC-style operations exposed by the generated backend client and consumed through the share-store.ts data layer. The table below summarises the main operations.

Operation	Purpose	Returns
createShare()	Validates input, uploads files, inserts the shares row, retries on code collision	Generated share code + link
uploadFile()	Uploads a single file to the share-files bucket under <CODE>/<relativePath>	Storage path + metadata
retrieveShare()	Calls the retrieve_share(code) RPC to fetch and account for one access	Content, files, remaining count
buildShareLink()	Builds the /r/<CODE> shareable URL for the QR code and copy button	Absolute share URL

The architecture diagram below places these pieces into the presentation, application, data, and storage layers described above.

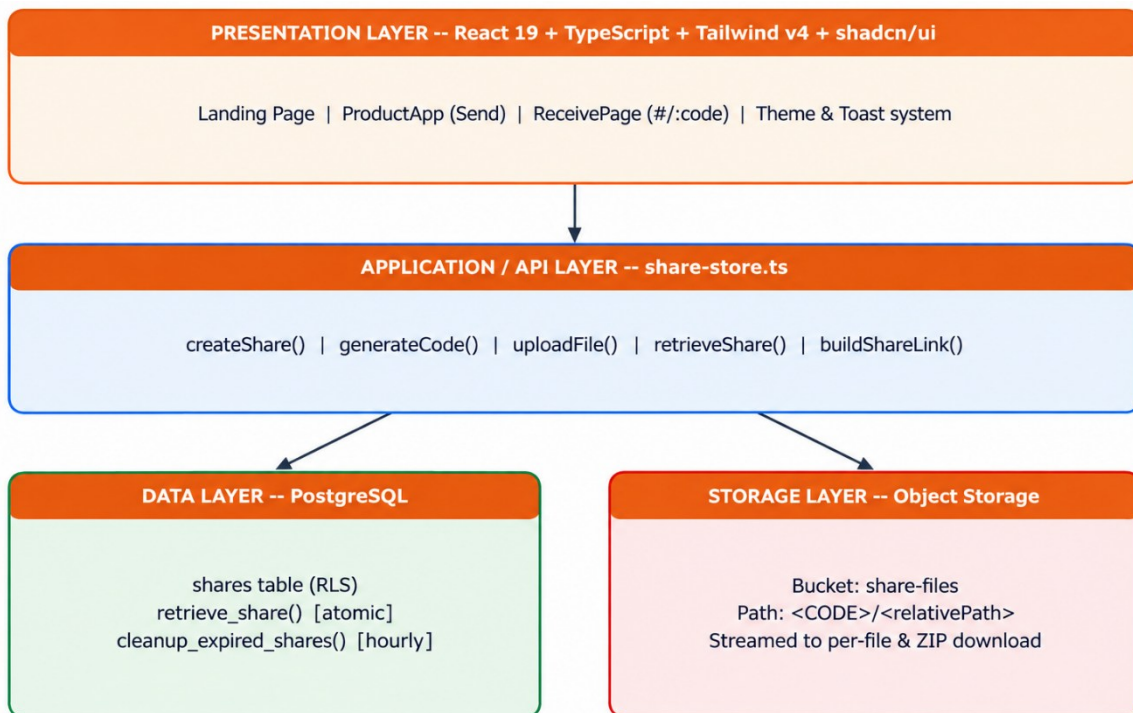


Fig 7.1 -- Three-Tier System Architecture

7.4 Security Architecture

- Row Level Security on the shares table blocks anonymous SELECT queries -- reads are only possible through the access-controlled retrieve_share() function.
- Row locking (SELECT ... FOR UPDATE) inside retrieve_share() prevents two simultaneous readers from both over-consuming a share's access limit.
- Only publishable, browser-safe keys are shipped to the client; no secret or service-role key exists in frontend code.
- The random share-code alphabet excludes visually ambiguous characters (0/O, 1/I), reducing accidental collisions from mistyped codes.
- Because reads always consume part of the access limit, a code cannot be brute-force enumerated into readable data without also exhausting the share it targets.

8. Implementation, Testing, Deployment & Evaluation Phase

8.1 Implementation

Implementation followed the increments defined in the Work Breakdown Structure (Section 5.2). The most significant implementation modules are summarised below.

Module	Responsibility
src/lib/share-store.ts	The entire data layer -- code generation, custom-code validation (<code>/^[A-Z0-9]{4,12}\$/</code>), file upload with rollback on failure, share creation with collision retry, retrieval, single-file download, and share-link building.
src/components/site/ProductApp.tsx	The interactive send/receive UI -- editor, file/folder pickers with size limits, expiry and access-limit selectors, random-vs-custom code toggle, and the generated-code card with copy/QR/link actions.
src/routes/r.\$code.tsx	The receive page -- fetches the share on mount, shows an expiry/remaining-access info bar, exposes Copy/Download actions above the snippet, and renders friendly not-found, expired, and exhausted states.
supabase/migrations/	SQL migrations defining the shares table, Row Level Security policies, the <code>retrieve_share()</code> and <code>cleanup_expired_shares()</code> functions, and storage bucket policies.

Constraints enforced throughout implementation include a 25 MB per-file limit, a 100 MB total-upload limit per share, a fixed 6-character random code length (4-12 characters for a custom code), and the fixed set of expiry presets listed in Section 4.2.

8.2 Testing

Testing combined manual end-to-end verification with browser automation, covering both the happy path and the edge cases that matter most for a self-destructing, access-limited system.

#	Test Case	Expected Result
1	Create a text-only share and retrieve it from a second browser context	Content appears identically on the receiving device
2	Create a share with multiple files and a nested folder	Folder structure survives the ZIP download
3	Retrieve a share whose <code>access_limit</code> is 1, then retrieve it again	Second attempt reports "exhausted"; row is deleted
4	Retrieve a share after its <code>expires_at</code> has passed	Attempt reports "expired"; row is removed by cleanup
5	Enter a custom code already in use	System returns a clear "code already in use" error
6	Two clients retrieve the same 1-access share simultaneously	Row locking allows exactly one successful retrieval

7	Toggle dark/light theme and hard-refresh the page	No colour flash; theme choice persists
8	Load the app on mobile, tablet and desktop widths	Layout remains fully usable at every breakpoint
9	Automated Playwright run of the send flow and receive page	Copy/Download buttons render above the snippet as expected

These cases were chosen to directly validate the functional requirements in Section 4.2 (especially FR-9 on auto-deletion and FR-11 on duplicate codes) and the non-functional reliability requirement around concurrent access.

8.3 Deployment

CodeDropz is deployed with a simple, fully automated pipeline: the GitHub repository is connected to Vercel, and every push to the default branch triggers a fresh build and deployment. The three environment variables the frontend needs -- the project URL, the publishable key, and the project ID -- are configured identically in both the local .env file and the Vercel project settings, and only publishable, browser-safe values are ever exposed to the client.

- Live application: <https://codedropz.vercel.app>
- Source repository: <https://github.com/LeviHQ/codedropz>
- Local development: npm install, followed by npm run dev (served at <http://localhost:8080>)
- Production build: npm run build, with deployment handled automatically by the connected Vercel project

8.4 Evaluation Phase

8.4.1 Objectives Achieved

- A genuinely cross-device, login-free transfer flow for text, code, files and whole folders.
- Fully self-destructing shares, enforced atomically at the database layer rather than trusted to the client.
- A polished, responsive, dark-by-default interface with no first-paint theme flash.
- A production deployment reachable from any modern browser, with SEO metadata and automated cleanup already in place.

8.4.2 Known Limitations

- Content is protected by code secrecy, the access limit and the expiry -- not by end-to-end encryption.
- There is no password-protection option for an individual share.
- There is no sender-side history or manual revoke, which was a deliberate trade-off to keep the product login-free.

- Upload limits (25 MB per file, 100 MB per share) are intentionally modest to keep the service fast and inexpensive to run.
- Dedicated rate limiting is not yet implemented; abuse prevention currently relies on platform-level limits.

8.4.3 Future Scope

- Client-side encryption with the key embedded in the share link's URL fragment, so the server never sees plaintext content.
- Password-protected drops with a server-side hash check.
- Syntax highlighting and automatic language detection on the receive page.
- Per-IP rate limiting for stronger abuse protection.
- A PWA build with Web Share Target support, so a phone can share directly into CodeDropz.
- Optional accounts for a private drop history, without breaking the existing anonymous path.
- A browser extension or CLI (codedropz push file.js) for developer workflows.

8.4.4 Conclusion

CodeDropz demonstrates a complete, production-deployed full-stack web application built on a modern React/TypeScript stack with a real relational backend, object storage, row-level security, and atomic server-side business rules. Across the problem statement, feasibility study, requirement engineering, SDLC planning, UML modelling, architecture design, and implementation/testing/deployment phases documented in this report, the project solves a small but very real everyday problem -- moving code and files between two devices -- with a design that prioritises speed, privacy through ephemerality, and zero onboarding friction.